ПР7. Генерация XML-документации по коду

Visual Studio помогает документировать такие элементы кода, как классы и методы, автоматически создавая стандартную структуру комментариев xml-документации. Во время компиляции можно создать XML-файл, содержащий комментарии документации.

XML-файл, созданный компилятором, можно распространять вместе со сборкой .NET, чтобы Visual Studio и другие интегрированные среды разработки могли использовать IntelliSense для отображения быстрых сведений о типах и членах. Вы также можете запустить XML-файл с помощью таких средств, как DocFX и Sandcastle , чтобы создать справочные веб-сайты API.

Включить сочетание клавиш для вставки комментариев можно, задав параметр "Примечания", чтобы автоматически вставлять структуры комментариев XML после ввода /// в C#. Для этого:

1) В строке меню Visual Studio выберите "Параметры инструментов>".
2) В диалоговом окне Параметры перейдите в Текстовый редактор>C# >Дополнительно.
3) В разделе "Комментарии" выберите Создать примечания к XML-документации для \\\.

Чтобы создать автодокументацию метода в Visual Studio, вам необходимо выполнить следующие шаги:

1) Откройте проект Visual Studio, содержащий метод, который вы хотите задокументировать.
2) Найдите метод и поместите курсор на имя метода.
3) В главном меню выберите Правка > IntelliSense > Создать комментарии XML-документации.
4) Это создаст шаблон для комментариев к документации метода, включая заполнители для имени метода, параметров и возвращаемого значения.
5) Заполните поля соответствующей информацией о методе, включая краткое описание того, что делает метод, имена и типы параметров метода и тип возвращаемого значения (если есть).
6) Сохраните изменения в файле и соберите проект.

Комментарии к документации будут включены в сгенерированную документацию для проекта, и их можно просмотреть с помощью средства просмотра документации Visual Studio или внешнего инструмента документации.

Структура комментариев XML немедленно создается над элементом кода. Например, при комментировании следующего GetUserName метода шаблон создает <summary> элемент, <param> элемент параметра и <returns> элемент для документирования возвращаемого значения.

```
/// <summary>
///
/// </summary>
/// <param name="id"></param>
/// <returns></returns>
public string GetUserName(int id)
{
    return "username";
}
```

Введите описания для каждого XML-элемента, чтобы полностью документировать код.

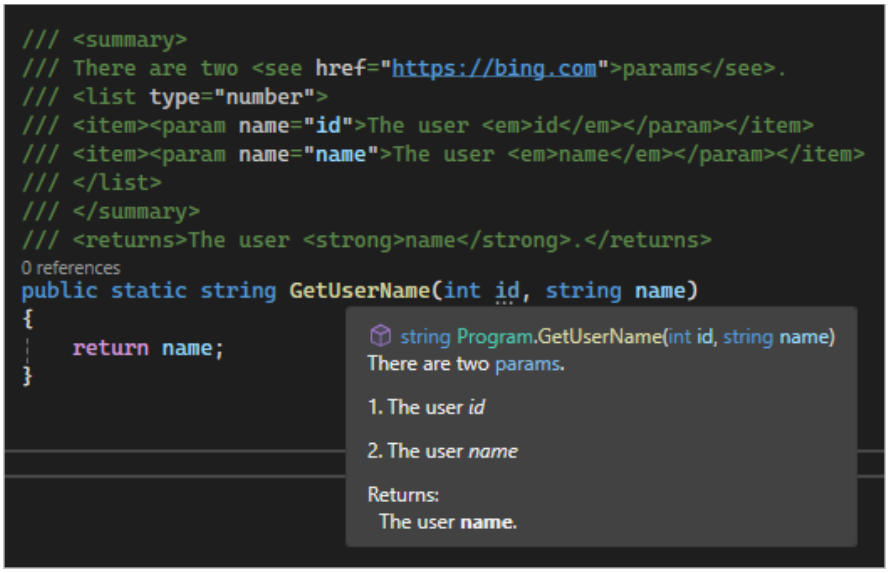
```
/// <summary>
/// Gets the username associated with the specified ID.
/// </summary>
/// <param name="id">The unique user ID.</param>
/// <returns>A string containing the username for the specified ID.</returns>
public string GetUserName(int id)
{
    return "username";
}
```

При наведении указателя мыши на код можно использовать XML-элементы и стили в комментариях, которые отображаются в кратких сведениях. К этим элементам относятся курсив или полужирные стили, маркированные или нумерованные списки, а также кликабельные ссылки cref или href.

Например, введите следующий код в файл программы C#:

```
/// <summary>
/// There are two <see href="https://bing.com">params</see>.
/// <list type="number">
/// <item><param name="id">The user <em>id</em></param></item>
/// <item><param name="username">The user <em>name</em></param></item>
/// </list>
/// </summary>
/// <returns>The <strong>username</strong>.</returns>
public static string GetUserName(int id, string name)
{
    return name;
}
```

При наведении указателя мыши на **GetUserName** панель "Быстрая информация" отображается следующим образом:



Примечание. Комментарии к XML-документации должны быть написаны в определенном формате и соответствовать определенным соглашениям, чтобы их можно было распознать и обработать в Visual Studio.

В таких XML-комментариях можно размещать XML-дескрипторы, содержащие документацию по типам и членам типов, используемым в коде.

XML-дескрипторы, распознаваемые компилятором, перечислены в следующей таблице:

Дескриптор	Описание
<c>	Помечает текст в строке как код
<code>	Помечает множество строк как код
<example>	Помечает пример кода
<exception>	Документирует класс исключения
<include>	Включает комментарии из другого файла документации
<list>	Вставляет список в документацию
<param>	Помечает параметр метода
<paramref>	Указывает, что слово является параметром метода
<permission>	Документирует доступ к члену
<remarks>	Добавляет описание члена
<returns>	Документирует возвращаемое методом значение
<see>	Представляет перекрестную ссылку на другой параметр
<seealso>	Представляет раздел "see also" ("смотреть также") в описании
<summary>	Представляет краткий итог о типе или члене
<value>	Описывает свойство

Практическая часть:

Чтобы увидеть, как это работает, рассмотрим пример кода, в который добавим некоторые XML-комментарии. Создадим новый консольный проект со следующим исходным кодом:

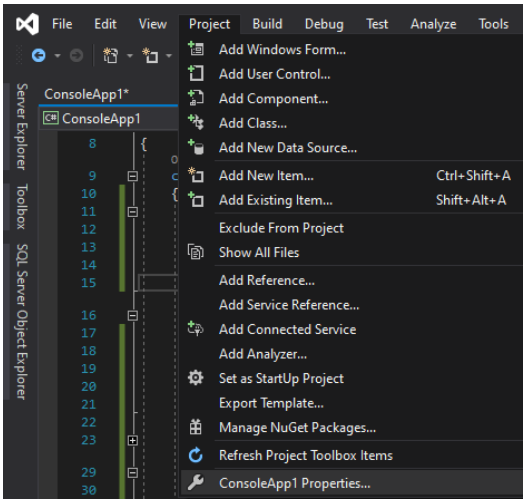
```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;

namespace ConsoleApplication1
{
    /// <summary>
    ///  Класс Program
    ///  основной класс программы
    ///  выводящий текст "Hello, World!"
    /// </summary>
    class Program
    {
        /// <summary>
        /// Метод Main() является
        /// входной точкой работы программы
        /// </summary>
        /// <param name="args">Аргументы метода Main()</param>
        static void Main(string[] args)
        {
            // Форматируем шапку программы
            Console.BackgroundColor = ConsoleColor.Green;
            Console.ForegroundColor = ConsoleColor.Black;
            Console.WriteLine("********************");
            Console.WriteLine("**** Мой проект ****");
            Console.WriteLine("********************");

            // Основная программа
            Console.BackgroundColor = ConsoleColor.Black;
            Console.ForegroundColor = ConsoleColor.Green;
            Console.WriteLine();
            Console.WriteLine("Hello, World!");

            // Ожидание нажатия клавиши Enter перед завершением работы
            Console.ReadLine();
        }
    }
}
```

Как создать файл документации

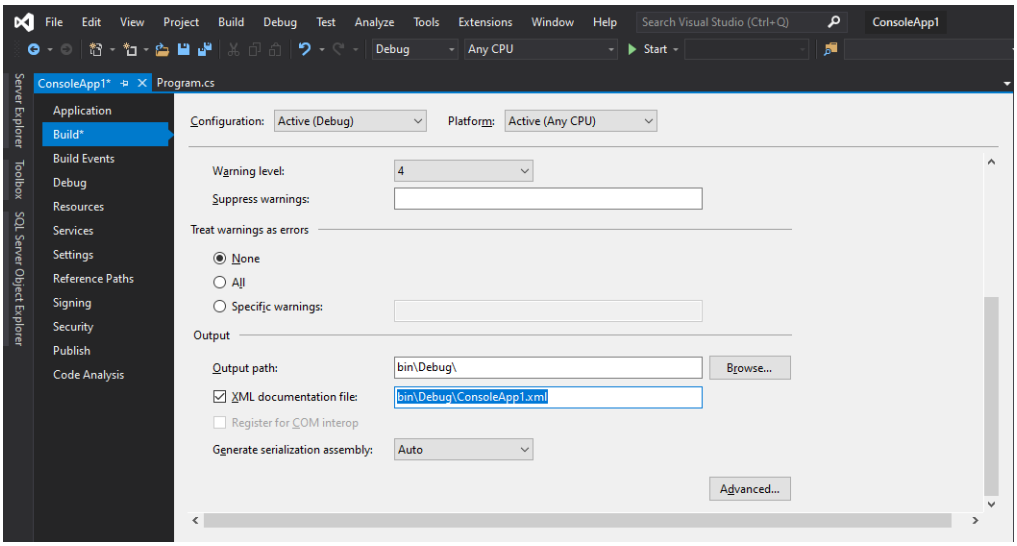
Иногда бывает нужно сохранить документацию вне кода. Чаще всего ее сохраняют в HTML-формате, а потом загружают на сайт, чтобы разработчики имели к ней доступ.

Способ 1

Для этого сначала нужно зайти в настройки проекта:



А потом перейти во вкладку Build и поставить галочку XML documentation file:



Способ 2

Либо можно изменить файл проекта, добавив следующее свойство внутри <PropertyGroup> элемента:

```
<GenerateDocumentationFile>True</GenerateDocumentationFile>
```

Способ 3

Также компилятор C# может извлекать XML-элементы из специальных комментариев и использовать их для генерации файлов XML. Для генерации XML-документации для сборки, указывается опция */doc* вместе с именем файла, который должен быть создан:

csc /t:library /doc:MyApplication.xml MyApplication.cs

Теперь вместе с компиляцией программы будет создаваться файл с документацией в формате XML.

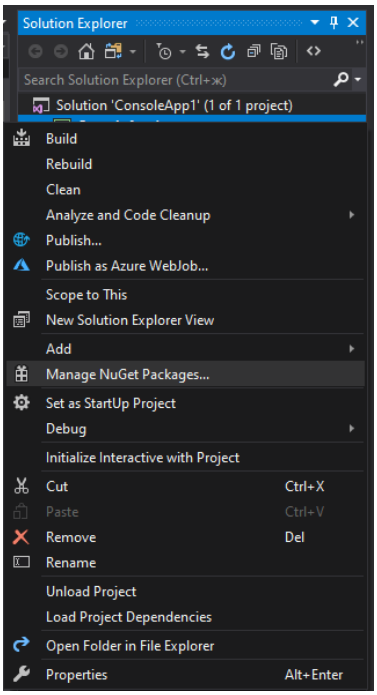
По итогу после сборки проекта в папке bin\Debug\net8.0\ будет сгенерирован XML-файл по имени MyApplication.xml со следующим содержимым:

```
<?xml version="1.0"?>
<doc>
    <assembly>
        <name>Program
    </assembly>
    <members>
        <member name="T:ConsoleApplication1.Program">
            <summary>
             Класс Program
             основной класс программы
             выводящий текст "Hello, World!"
            </summary>
        </member>
        <member name="M:ConsoleApplication1.Program.Main(System.String[])">
            <summary>
            Метод Main() является
            входной точкой работы программы
            </summary>
            <param name="args">Аргументы метода Main()</param>
        </member>
    </members>
</doc>
```

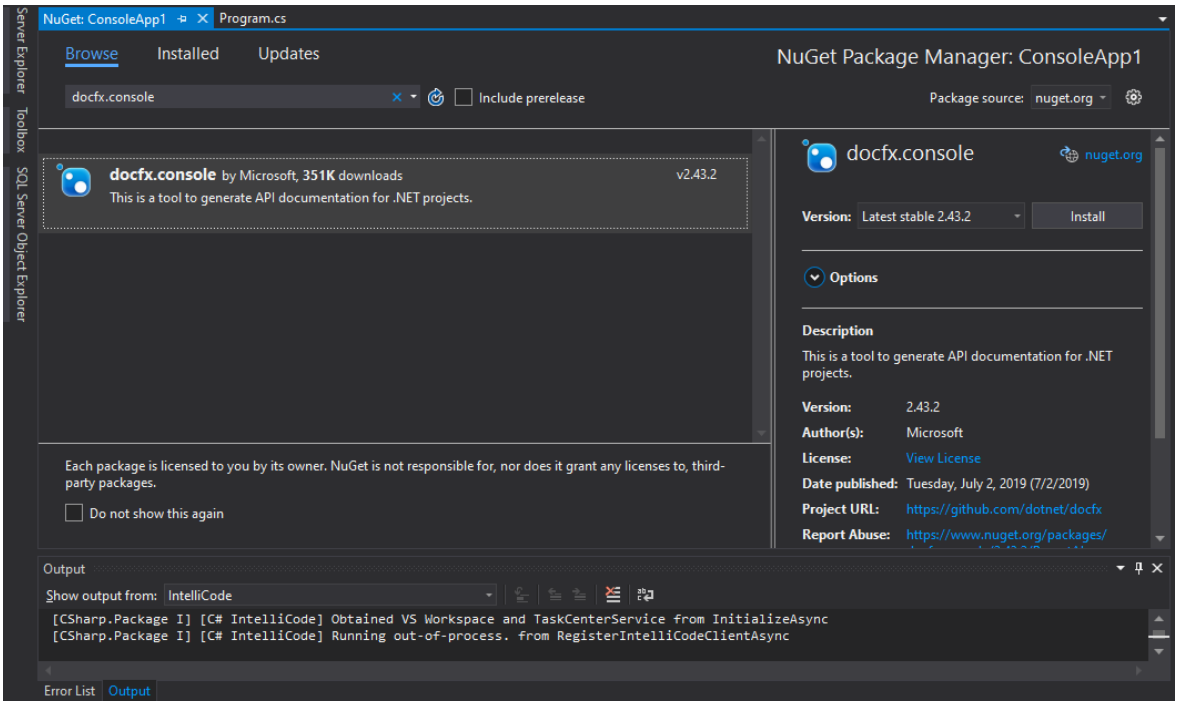
Обратите внимание на то, что компилятор на самом деле выполнил некоторую работу за вас: он создал элемент <assembly> и также добавил элементы <member> для каждого члена класса в этом файле. Каждый элемент <member> имеет атрибут name с полным именем члена, снабженным префиксом - буквой, который указывает на то, является он *типом* (T:), *полем* (F:) или *членом* (M:).

Такой файл можно преобразовать в HTML с помощью специальных утилит. Microsoft для этого рекомендует использовать DocFX или Sandcastle.

Создадим новый проект и рассмотрим преобразование на примере DocFX. Его можно скачать с помощью NuGet Package Manager в Visual Studio. Для этого нажмите на проект правой кнопкой мыши и выберите пункт Manage NuGet Packages:



Затем перейдите во вкладку Browse и введите в поле поиска название docfx.console, а потом нажмите Install:

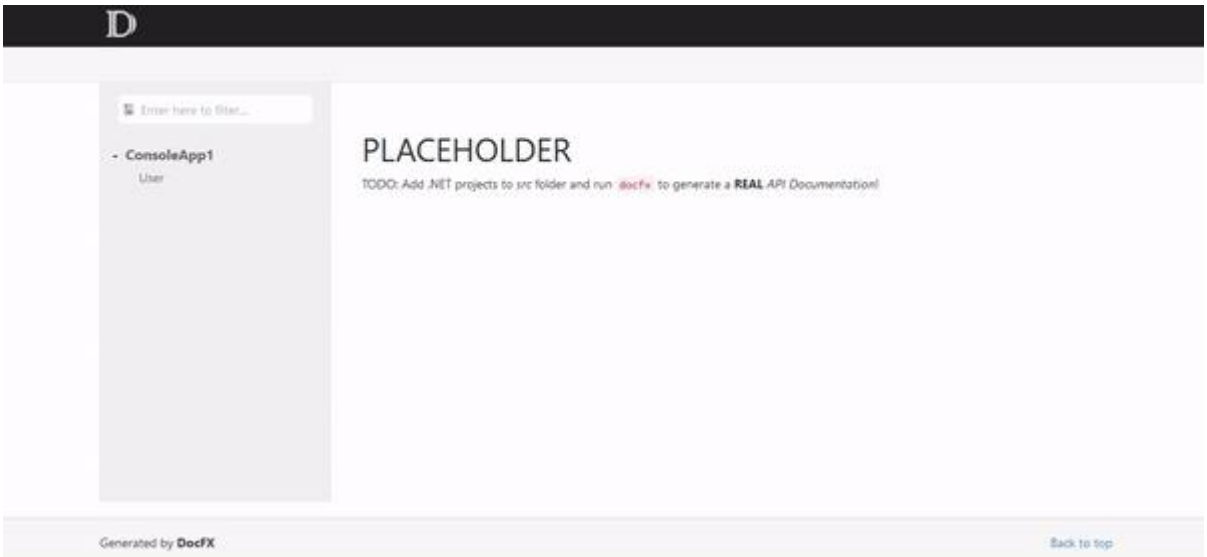


После нужно подтвердить установку и согласиться с условиями лицензионного соглашения.

Теперь при сборке проекта будет создаваться папка _site, в которой находится сайт с документацией. Однако это касается класса Program, поэтому чтобы проверить работу DocFX, нужно добавить какой-нибудь класс:

```
namespace ConsoleApp1
{
	/// <summary>
	/// Класс, который представляет пользователя.
	/// </summary>
	public class User
	{
		private string name;

		/// <summary>
		/// Конструктор класса.
		/// </summary>
		/// <param name="name">Имя пользователя.</param>
		public User(string name)
		{
			this.name = name;
		}

		/// <summary>
		/// Меняет имя пользователя.
		/// </summary>
		/// <param name="name">Имя пользователя.</param>
		public void ChangeName(string name)
		{
			this.name = name;
		}
```

```
            /// <summary>
            /// Метод для вывода имени пользователя.
            /// </summary>
            /// <returns>Возвращает имя пользователя.</returns>
            public string GetName()
            {
                    return this.name;
            }

        }
}
```

После компиляции проекта с таким классом можно проверить сайт. Его главная страница будет пуста — она нужна для того, чтобы вкратце описать свою программу. Сама же документация находится по адресу api/index.html.

Вот как она выглядит: